Lecture 59 : Software Reliability - III

Introduction

In the complex world of software development, delivering a product that consistently performs as expected is paramount. Yet, accurately predicting how reliable a piece of software will be in real-world scenarios presents a unique challenge, far more intricate than estimating the reliability of physical systems. Unlike hardware, software doesn't wear out or degrade; its failures stem from design flaws or incorrect logic, which are often latent until specific conditions trigger them. The previous lecture touched upon the inherent difficulties in software reliability measurement and introduced some basic metrics. This lecture delves deeper into the practical methodologies for quantifying software reliability, focusing on two primary techniques: reliability growth modeling and statistical testing. Understanding these approaches is crucial for project managers and developers alike, as they provide the tools to not only estimate current reliability but also to predict future performance and optimize testing efforts, ultimately leading to more robust and dependable software systems.

Core Concepts

Term	Definition	Significance
`Reliability Growth Modelling`	A graphical or mathematical approach that tracks how software reliability improves over time as errors are detected and fixed during testing or operation.	Allows prediction of future reliability and estimation of required testing duration to meet specific reliability targets.
`Statistical Testing`	A testing methodology where test cases are designed based on the software's operational profile, aiming to estimate reliability rather than solely to find defects.	Provides a quantitative, user-centric reliability value that reflects real-world usage patterns, making it agreeable to most stakeholders.
`Operational Profile`	A quantitative description of how a software system is expected to be used, detailing the frequency and probability of	Essential for statistical testing, as it ensures test data accurately represents typical user behavior,

Term	Definition	Significance
	different functions and scenarios being invoked.	leading to more realistic reliability estimates.
`Step Function Model`	A simplistic reliability growth model assuming that each error fix ideally and constantly increases software reliability, without introducing new errors.	Serves as a foundational, intuitive model, though its oversimplified assumptions limit its practical accuracy for complex software.
`Jelinski Moranda Model`	A reliability growth model where the failure rate is assumed to be proportional to the number of remaining faults, implying varying reliability improvements per fix.	An improvement over the step function model by acknowledging non-constant reliability growth, but still has limitations regarding the impact of early vs. late bug fixes.
`Littlewood and Verall's Model`	A sophisticated reliability growth model that accounts for negative reliability growth (when fixes introduce new errors) and diminishing returns from bug fixes, treating bug contributions as random variables.	Offers a more realistic and accurate representation of reliability growth by aligning with intuitive understanding of how bug fixes impact overall system dependability.
`Execution Time`	The actual CPU time or processing time during which the software is actively running and performing computations, excluding idle time waiting for user input.	Crucial for accurate reliability estimation in statistical testing, as using clock time can significantly skew results due given the intermittent nature of human interaction.

Detailed Analysis

1. Software Reliability Estimation Techniques

Estimating the reliability of a software application, once developed and deployed, is a critical task that requires quantitative methods. The goal is to assign a numerical

value to the application's reliability. This lecture primarily focuses on two main techniques: reliability growth modeling and statistical testing. While indirect methods, which predict reliability based on software characteristics like complexity metrics, exist, they will not be covered in detail here.

2. Reliability Growth Modelling

Reliability growth modeling is a technique used to graphically represent and predict how the reliability of software evolves over time. As errors are detected and subsequently repaired, the software's reliability is expected to improve. This model helps in understanding this growth pattern.

The primary uses of a reliability growth model include:
* **Predicting Future Reliability**: Estimating what the software's reliability will be after a certain period of operation or further testing.
* **Estimating Testing Duration**: Determining how much longer testing needs to continue to achieve a predefined quantitative reliability requirement. Since testing uncovers bugs that are then fixed, the model can predict the necessary testing effort.

The Step Function Model

The step function model is one of the oldest and simplest reliability growth models. Its core assumptions are:
* **Error Detection and Fixation**: Each time a failure occurs, the underlying error is detected and immediately fixed.
* **Decreasing Errors, Increasing Reliability**: Fixing errors reduces the total number of errors in the software, thereby increasing its reliability.
* **Ideal Fixes**: All error fixes are assumed to be perfect, meaning they permanently resolve the issue without introducing any new side effects or bugs.
* **Constant Reliability Increase**: Each error fix is assumed to increase reliability by a constant, uniform amount.

Graphically, this model shows the rate of occurrence of failure decreasing in steps over time as fixes are applied. However, its simplicity is also its major limitation. Real-world software reliability growth is rarely constant. Bugs in core functionalities, when fixed, often lead to a much larger increase in reliability compared to fixes in less critical or remote parts of the software. Therefore, this model's results are often not very accurate due to its oversimplified assumptions, particularly that all errors contribute equally to reliability growth.

The Jelinski Moranda Model

Proposed long after the step function model, the Jelinski Moranda model offers a more refined perspective. Its fundamental assumption is that the failure rate of the software is directly proportional to the number of faults remaining within it. This implies that as more faults are removed, the failure rate decreases.

Key characteristics of this model:
* **Proportional Failure Rate**: If `n` is the number of faults, the failure rate is `k * n` (where `k` is a constant). After one bug is fixed, the failure rate becomes `k * (n-1)`.
* **Non-Constant Reliability Improvement**: Unlike the step function model, the improvement in reliability per bug fix is not constant. Initially, when `n` is large, fixing one bug (out of `n`) leads to a relatively smaller proportional improvement (e.g., `1/n`). Towards the end, when `n` is small (e.g., 2 bugs remaining), fixing one bug leads to a much larger proportional improvement (e.g., `1/2`).

While an improvement, this model still has shortcomings. Intuitively, bugs in the core parts of the software are often discovered early and their fixes contribute significantly to reliability growth. As testing progresses, the remaining bugs tend to be in less frequently used or peripheral parts, and their fixes yield diminishing returns in terms of overall reliability improvement. The Jelinski Moranda model, however, suggests that the reliability growth rate would be higher towards the end, which contradicts this intuitive understanding. It does not accurately reflect the diminishing returns observed in real-world software testing.

The Littlewood and Verall’s Model

Addressing the limitations of earlier models, the Littlewood and Verall’s model introduces more realistic assumptions. A critical improvement is its acknowledgment that bug fixes are not always ideal. In reality, fixing one bug can sometimes introduce new errors, leading to a temporary decrease in reliability, or "negative reliability growth."

Key features of this model:
* **Accounts for Negative Reliability Growth**: It recognizes that a repair might introduce further errors, causing reliability to decrease rather than increase.
* **Variable Reliability Improvement**: The average improvement in reliability per repair is not constant and can decrease over time, reflecting the possibility of negative growth.
* **Random Variable Contribution**: The model treats a corrected bug's contribution to reliability as an independent random variable, often following a gamma distribution.
* **Diminishing Returns**: It aligns with the intuitive understanding that bugs with

larger contributions to reliability (often those in core functionalities) are detected and fixed earlier, leading to higher initial reliability growth. Later fixes, for less impactful bugs, result in smaller reliability gains. This represents diminishing returns as testing continues.

By incorporating these more realistic aspects, the Littlewood and Verall’s model provides a more accurate approximation of software reliability growth compared to the step function and Jelinski Moranda models.

Selecting a Reliability Growth Model

With dozens of reliability growth models available, each with its own set of simplifying and realistic assumptions, choosing the "best" model is not straightforward. There isn't a single model universally applicable to all software applications, as reliability growth is highly dependent on factors like bug distribution, software complexity, and the number of functions supported.

The practical approach to model selection involves:
1. **Collecting Failure Data**: Gathering considerable failure data over a period of time during testing or operation.
2. **Plotting and Matching**: Plotting this observed failure data against various growth models.
3. **Best Fit**: Identifying the model that most accurately fits the observed data. This "best fit" model is then used to predict future reliability or estimate further testing duration.

3. Statistical Testing

Statistical testing is a distinct testing process whose primary objective is to determine the reliability of software, rather than solely to detect defects. While defects may be uncovered during this process, finding them is not the main focus. The key differentiator of statistical testing is its test case design, which is fundamentally different from traditional defect testing.

Operational Profile Definition

The foundation of statistical testing is the software's operational profile. This profile is crucial because software reliability is often "observer-dependent"; different users might perceive different levels of reliability based on their usage patterns. To provide a single, universally agreeable reliability value, statistical testing bases its approach on the average usage patterns of all users.

Steps to define the operational profile:
1. **Identify Functionalities and Scenarios**: List all functions supported by the

software (e.g., "create document," "edit document," "print file" for a word processor) and their various scenarios.
2. **Capture User Patterns**: Monitor actual user behavior, perhaps by deploying log-collecting software, to understand how frequently different functions and scenarios are invoked.
3. **Assign Probabilities**: Based on the collected usage logs, assign a probability value to each input class or scenario. This probability represents the likelihood of that function or scenario being invoked by an average user. This collection of functionalities and their associated probabilities constitutes the operational profile.

Test Data Generation

Once the operational profile is defined, the next step is to generate test data that accurately reflects this profile. This means the distribution of test cases should mirror the probabilities defined in the operational profile.

For example:
* If a specific scenario (e.g., `f11` of `F1` functionality) has a probability of 0.4 (40%) in the operational profile, then 40% of the test cases in the test suite should correspond to that `f11` scenario of `F1`.
* Similarly, if another scenario (`f12` of `F1`) has a probability of 0.1 (10%), then 10% of the test cases should target that `f12` scenario.

This approach contrasts sharply with defect testing, where test cases are often designed to target anticipated defect-prone areas or edge cases, without necessarily reflecting typical user behavior. In statistical testing, the test data is generated directly from the user's expected usage pattern.

Execution and Measurement

After generating the test data, the software is executed using these test cases. A critical aspect of this step is the measurement of time. It is essential to measure the actual **execution time** of the software, rather than just the clock time.

Why execution time is crucial:
* Software often completes functionalities in milliseconds but then idles for seconds or minutes awaiting human input.
* Using clock time would inaccurately inflate the "time" during which the software is operational, leading to a very inaccurate reliability estimation.
* Instrumentation or specific means are required to collect the actual run-time information, which is then used as the time of failure for reliability calculations.

Reliability Computation

After a statistically significant number of failures have been observed during the execution phase, the software's reliability can be computed. This involves analyzing the number of functionalities invoked versus the number of failures observed. Because the test data is derived directly from the operational profile, the resulting reliability value is likely to be agreeable to most users, as it reflects how the software performs under typical usage conditions. This method provides a quantitative, user-centric measure of software dependability.

Key Takeaways

- [ ] Software reliability estimation is more challenging than hardware reliability due to the nature of software defects.
- [ ] Reliability growth modeling tracks how software reliability improves as errors are fixed over time.
- [ ] Reliability growth models can predict future reliability and estimate the required duration of testing.
- [ ] Simple models like the Step Function and Jelinski Moranda models have limitations due to oversimplified assumptions about error fixes and reliability growth.
- [ ] More advanced models, such as Littlewood and Verall’s, account for realistic scenarios like negative reliability growth and diminishing returns from bug fixes.
- [ ] Selecting the appropriate reliability growth model involves fitting observed failure data to various models to find the best match.
- [ ] Statistical testing aims to determine software reliability by designing test cases based on the software's operational profile.
- [ ] The operational profile quantifies average user behavior, ensuring that reliability estimates reflect real-world usage.
- [ ] Accurate reliability measurement in statistical testing requires tracking actual execution time, not just clock time, to account for software idling.
- [ ] The reliability value derived from statistical testing is generally well-accepted because it is grounded in realistic user interaction patterns.

Conclusion

This lecture has illuminated the critical methodologies for quantifying software reliability, moving beyond qualitative assessments to provide concrete, measurable insights. Reliability growth modeling offers a dynamic view of how software quality evolves during development and testing, enabling project managers to forecast

future performance and optimize resource allocation for testing. By understanding the nuances of models like the Step Function, Jelinski Moranda, and Littlewood and Verall’s, we gain a deeper appreciation for the complexities involved in accurately predicting reliability, especially as software systems become more intricate.

Complementing this, statistical testing provides a user-centric approach, grounding reliability estimates in the actual operational profile of the software. This method ensures that the reported reliability is not just a theoretical number but a reflection of how the software performs under typical usage conditions, making it highly relevant to end-users and stakeholders. Together, these techniques empower development teams to make informed decisions, set realistic reliability targets, and ultimately deliver software that is not only functional but also consistently dependable in the hands of its users. As software continues to permeate every aspect of our lives, the ability to accurately estimate and manage its reliability will remain a cornerstone of successful engineering and project management.